- *free*
  GNU/Linux provides the *free* command to check memory usage. It displays the total amount of free and used physical and swap memory in the system, as well as the buffers and caches used by the kernel. Shown below is a sample output of the *free* command:

```
$ free
total     used      free      shared  buff/cache   available
Mem:  8117768  1267836   3718996   153112  3130936    6393176
Swap:       2097148            0     2097148
```

  In the above output:
  -- 'total' stands for the total installed memory on current system
  -- 'used' stands for the used memory. It is calculated as follows: [total - (free + buffers + cache) memory]
  -- 'free' stands for unused memory
  -- 'shared' stands for the shared memory used by tmpfs
  -- 'buffers' stands for the memory used by kernel buffers
  -- 'cache' stands for the memory used by the page cache and slabs
  -- 'buff/cache' stands for the sum of the buffers and cache memory
  -- 'available' stands for an estimation of how much memory is available for starting new applications, without swapping.

- *du*
  As the name suggests, the *du* command is used to calculate disk usage. It summarises disk usage of the set of files/directories.
  To calculate the size of the directory, execute the command given below:

```
$ du -sh DIR-PATH
```

  In the above example:
  -- option *s* is used to display only a total for each argument
  -- option *h* is used to show the output in human readable format (K for KB, M for MB and so on)

- *df*
  As the name suggests, the *df* command is used to get information about free disk space. It reports the file system's disk space usage. If no file name is given, the space available on all currently mounted file systems is shown:

```
$ df -h
```

  In the above example:
  -- option *h* is used to show output in a human readable format (K for KB, M for MB and so on)

In this article, we have discussed some of the popular GNU/Linux utilities briefly. Mastering these utilities will take your knowledge to the next level. To know more about each utility, do refer to the official documentation. END

By: Narendra K.
The author is a FOSS enthusiast. He can be reached at *narendra0002017@gmail.com.*

Continued from page...60

Spark can be deployed as a standalone cluster (if paired with a capable storage layer) or can hook into Hadoop as an alternative to the MapReduce engine.

Apache Flink

Apache Flink is an open source platform; it is a streaming data flow engine that provides communication, fault tolerance and data distribution for distributed computations over data streams. It is a scalable data analytics framework that is fully compatible with Hadoop. Flink can execute both stream processing and batch processing easily.

While Spark performs batch and stream processing, its streaming is not appropriate for many use cases because of its micro-batch architecture. Flink's stream-first approach offers low latency, high throughput, and real entry-by-entry processing.

Table 1 Compares the features of Hadoop, Storm, Samza, Spark and Flink. END

References

[1] *https://www.forbes.com/sites/bernardmarr/2015/06/22/spark-or-hadoop-which-is-the-best-big-data-framework/*
[2] *https://www.kdnuggets.com/2016/03/top-big-data-processing-frameworks.html*
[3] *https://www.digitalocean.com/community/tutorials/hadoop-storm-samza-spark-and-flink-big-data-frameworks-compared*
[4] *http://hadoop.apache.org/*

By: Prof. Madhuri Chopade and Prof. Dulari Bhatt
Prof. Madhuri Chopade is an assistant professor in the IT department at Gandhinagar Institute of Technology (affiliated to GTU). She can be contacted at *madhuri.chopade@git.org.in.*
Prof. Dulari Bhatt is an assistant professor in the IT department at Gandhinagar Institute of Technology. She can be contacted at *dulari.bhatt@git.org.in.*